Diff-Eval

An AI-Driven Agentic Workflow for Code Quality Evaluation

Jiahe (Jay) Chen

Computation and Design (Computer Science)
Duke Kunshan University

Mentor: Prof. Jiacheng Shen, Assistant Professor of ECE

March 2026

昆山杜克大学
DUKE KUNSHAN UNIVERSITY

Introduction: The Problem

"Code quality is a critical concern in software engineering, directly affecting system stability, maintainability, and development efficiency."

What Static Analyzers Do

- Rule-based pattern matching (SonarQube, Checkstyle, PMD)
- Fast and reliable for syntactic/structural issues

Where They Fall Short

- Cannot understand *architectural intent*
- Fail to detect **design-level** violations (e.g. SOLID)
- Require understanding *why* code is structured the way it is

Background: SOLID Principles

SOLID (R. C. Martin, 2000)

S	Single Responsibility — one reason to change
O	Open / Closed — open for extension, closed for modification
L	Liskov Substitution — subtypes replaceable for base types
I	Interface Segregation — no forced unused dependencies
D	Dependency Inversion — depend on abstractions, not concretions

Why SOLID Matters

- Violations produce **rigid, fragile, tightly coupled** code
- Resist future modification and raise maintenance costs
- Hard to detect automatically — requires understanding *intent*, not just syntax

The Detection Challenge

Static analyzers catch *syntax*. SOLID violations are *structural* and *semantic* — they require reasoning about architectural design intent.

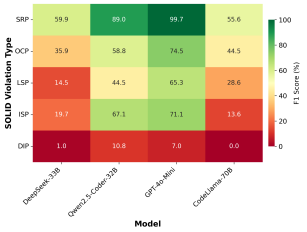
Background: LLM-Based Detection & Benchmark Dataset

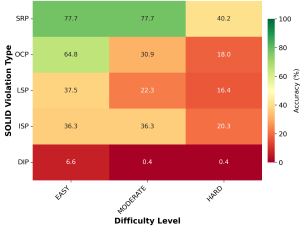
Pehlivan et al. (2025)

- First systematic study: 4 models, 4 prompt strategies
- Best: `example` (few-shot) strategy
- GPT-4o-mini peaks at **69.2%**; open-source $<25\%$
- DIP essentially unsolved: F1 = 0–10.8
- Qwen family top open-source performer $\Rightarrow$ motivates model choice

Benchmark Dataset

- 240 examples, 48 per SOLID principle
- 4 languages: Python, Java, Kotlin, C#
- Easy (33 LOC) / Moderate (140 LOC) / Hard (306 LOC)

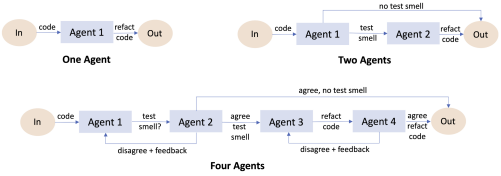




Background: Agentic Workflows & Baselines

Melo et al. (2025) — Test Smell Detection

- Applied agentic workflows to code quality with small models (3B–14B)
- Detector–Evaluator feedback loop outperforms single-pass
- Phi-4-14B (4 agents) within 5% of single-agent GPT-4o-mini



Our Baselines — both use Qwen3.5 9B

Single Agent

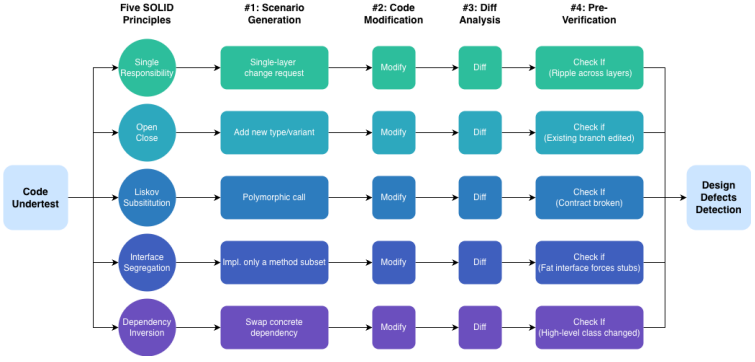
- Replicates Pehlivan `example` strategy
- Code in → violated principle out

Two-Agent (Detector + Evaluator)

- Agent 1 detects with reasoning
- Agent 2 critiques & refines (up to 3 rounds)
- Adopts Melo et al. pattern

- Same model across all three workflows
- Differences ⇒ **workflow design**, not model scale

Diff Eval: System Design



Core Design: simulates real software development scenarios by modifying the code proactively to test the extendability

Scenario Gen.
Principle-specific extension task engineered to trigger violation

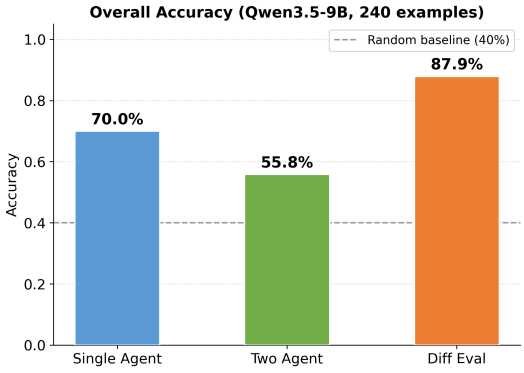
Code Modification
LLM implements the scenario on the original code

Diff Analysis
Unified diff computed via Python `difflib`

Pre-Verification
Diff pattern matched to principle's violation signal

Unified Ranking
Aggregates 5 verdicts → Top-N ranked output

Results: Overall Performance

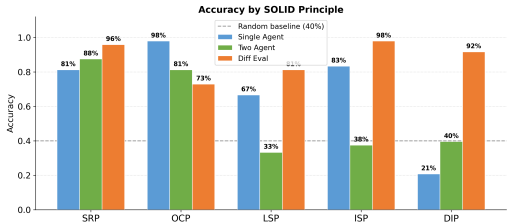


Method	Acc.	MRR@2	F1
Single Agent	70.00%	0.638	0.699
Two-Agent	55.83%	0.490	0.576
Diff Eval	**87.92%**	**0.792**	**0.888**

- **+17.9 pp** over Single Agent
- **+32.1 pp** over Two-Agent
- Macro-F1 0.888 — balanced across all five principles
- Two-Agent *underperforms* SA: debate without better evidence degrades accuracy
- *Metric*: Top-2 Accuracy — ground-truth violation appears in model's top-2 predictions

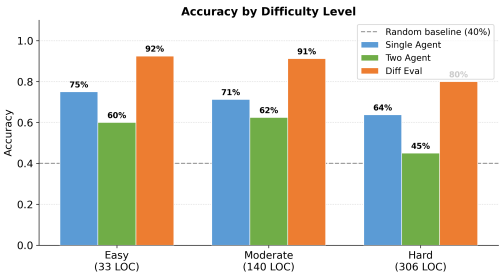
Results: By Violation Type & Difficulty Level

By SOLID Principle



- **DIP**: 20.83% (SA) → **91.67%** (Diff Eval)
  Tight coupling invisible statically; concrete in diffs
- **ISP**: 83.33% → **97.92%**
- **OCP**: SA leads (97.92% vs. 72.92%)
  Surface `if-else` — direct inspection wins

By Difficulty Level



- Diff Eval leads at **every** difficulty level
- Easy→Hard drop: **12.5 pp** vs. 11.25 pp (SA) / 15 pp (TA)
- Hard: **80.00%** vs. 63.75% (SA) / 45.00% (TA)
- Pre-Verification passes only the compact diff
  ⇒ insulates ranking from long-code noise

Results: Comparison with Full-Size Cloud LLM

System	Top-2 Accuracy	MRR@2
Qwen3.5 9B — Single Agent (local)	70.00%	0.638
Qwen3.5 9B — Diff Eval (local)	**87.92%**	**0.792**
Qwen3.5 Plus — Single Agent (cloud)	95.83%	0.858

Key Finding

- Plain 9B baseline lags cloud by **25.83 pp**
- Diff Eval recovers **17.92 pp** through workflow design alone
- Closes ∼**70%** of gap — no increase in model size
- **Zero data egress**

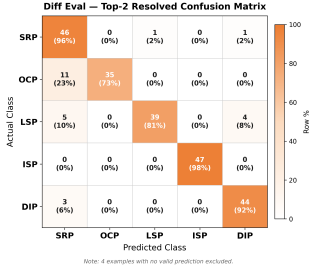
Why it matters

- Regulated code cannot leave the premises
- Consumer hardware: RTX 4060 8GB, i5-12400F
- On DIP specifically: Diff Eval **matches** Qwen3.5 Plus (91.67%)
- Entire pipeline is **observable**: every diff, verdict, and ranking rationale is logged

Discussion

Error Analysis — SRP Over-prediction

- 29 total errors; **19** are SRP over-prediction
- OCP→SRP (11), LSP→SRP (5), DIP→SRP (3)
- Responsibility-mixing symptoms appear across violation types
  ⇒ conceptual overlap inherent in SOLID taxonomy



Why Diff Eval Works

- Abstract judgment → **concrete evidence**
- DIP & ISP: invisible statically, concrete in diffs
- Pre-Verification compresses context for robustness

Limitations

- **OCP** (72.92%): SRP scenario absorbs OCP signals
- **Runtime**: 20 calls/example (151.6 s) vs. 1 call (2.1 s)
- **Dataset**: 240 examples only

Observability & Privacy

- Every artifact persisted and inspectable
- Transparent reasoning — not a black box
- Compliant: EU AI Act, Katakri 2020

Conclusion

Small LLM + Agentic Workflow ≈ Full-Size LLM

in the code quality evaluation scenario

Results

- **87.92% accuracy** on SOLID detection using a local 9B model
- Closes ∼**70%** of gap to Qwen3.5 Plus (95.83%) through workflow design alone
- Consumer hardware, **zero data egress**
- +17.9 pp over single-agent; balanced across all five principles
- Design quality best revealed by **behavioral observation** (what happens when you extend the code), not static judgment

Future Work

1. Hybrid: best method per principle
2. Improved OCP scenario generation
3. Larger-scale evaluation on real codebases

昆山杜克大学
DUKE KUNSHAN UNIVERSITY

Jiahe (Jay) Chen

Mentor: Prof. Jiacheng Shen
Duke Kunshan University
Class of 2026

References

1. R. C. Martin, “Design Principles and Design Patterns,” 2000.
2. R. C. Martin, *Clean Architecture*, Prentice Hall, 2017.
3. Pehlivan et al., “Evaluating LLMs for SOLID Violation Detection,” *arXiv*, 2025.
4. Melo et al., “Agentic LLM Workflows for Test Smell Detection,” *arXiv*, 2025.
5. Sharma et al., “Designite: A Software Design Quality Assessment Tool,” *ICSE*, 2018.
6. Palomba et al., “Detecting Bad Smells Using Change History,” *ASE*, 2015.
7. Wang et al., “A Survey on LLM-Based Autonomous Agents,” *Front. Comput. Sci.*, 2024.
8. Yao et al., “ReAct: Synergizing Reasoning and Acting in LLMs,” *ICLR*, 2023.
9. LangChain Inc., *LangGraph*, 2024.
10. Ollama, *Local LLM Inference Framework*, 2024.
11. Qwen Team, “Qwen2.5 Technical Report,” *arXiv*, 2025.
12. European Parliament, *EU AI Act*, 2024.
13. Al-Omar et al., “On the Relationship Between Code Smells and SOLID,” *JSS*, 2023.